

RiffBanks

Applicazioni e Servizi Web

Lorenzo Leoni - 0001192033
`lorenzo.leoni5@studio.unibo.it`

Anno Accademico 2025/2026

Indice

1	Introduzione	3
2	Requisiti	4
2.1	Analisi dell'utenza	4
2.2	Requisiti Utente	4
2.3	Requisiti Funzionali	4
2.3.1	Autenticazione e Gestione Utente	4
2.3.2	Gestione delle Band	5
2.3.3	Progetti Musicali	5
2.3.4	Comunicazione	5
2.3.5	Modulo Gig Economy	5
2.4	Requisiti Non Funzionali	5
2.4.1	Usabilità	5
2.4.2	Accessibilità (WCAG 2.1 livello AA)	6
2.4.3	Sicurezza	6
2.4.4	Performance e Scalabilità	6
3	Design	7
3.1	Metodologia	7
3.2	Architettura delle Interfacce Utente	8
3.2.1	Design System e Fonti di Ispirazione	8
3.2.2	Processo di Creazione dei Mockup	8
3.2.3	Mockup	9
3.2.4	Behavior-Driven Development	11
3.2.5	Viste dell'Applicazione	12
3.2.6	Screenshot dell'Applicazione	12
4	Tecnologie	15
4.1	Containerizzazione	15
4.2	Backend	15
4.2.1	Runtime e Framework	15
4.2.2	Persistenza Dati	15
4.2.3	Real-time	15
4.2.4	Autenticazione e Sicurezza	16
4.2.5	Upload File	16
4.3	Frontend	16
4.3.1	Framework e Architettura	16
4.3.2	Build e Sviluppo	16
4.3.3	UI e Styling	16
5	Codice	17
5.1	Struttura del Progetto	17
5.1.1	Client	17
5.1.2	Server	17
5.1.3	Diagramma dei Componenti	18
5.2	Pattern e Implementazioni Rilevanti	18
5.2.1	Controller Backend: Gestione Band	18

5.2.2	State Management Frontend: Auth Store	19
5.2.3	Real-time: Socket.io	20
5.2.4	Sistema di Voto	20
5.2.5	Diagramma Comunicazione Real-time	21
6	Test	22
6.1	Test Tecnici	22
6.2	Euristiche di Nielsen	22
6.3	Test di Usabilità	23
6.4	Validazione e Accessibilità	23
7	Deployment	25
7.1	Architettura dei Container	25
7.2	Build Multi-Stage del Frontend	25
7.3	Configurazione Nginx	25
7.4	Persistenza dei Dati	26
7.5	Variabili d'Ambiente	26
7.6	Health Checks	27
7.7	Avvio dell'Applicazione	27
7.8	Dipendenze tra Servizi	27
8	Conclusioni	28
8.1	Sviluppi Futuri	28
8.2	Commenti Finali	28

Capitolo 1

Introduzione

Nel panorama musicale contemporaneo, i musicisti e le band si trovano quotidianamente ad affrontare una sfida organizzativa significativa: la gestione della collaborazione attraverso strumenti frammentati e disconnessi tra loro. Le conversazioni avvengono su WhatsApp o Telegram, i file audio vengono condivisi tramite Google Drive o WeTransfer, il coordinamento delle sessioni passa attraverso email, e la ricerca di nuovi collaboratori richiede la consultazione di molteplici piattaforme e gruppi social. Questa frammentazione genera inefficienze considerevoli, dalla difficoltà nel reperire versioni specifiche di un brano alla perdita di contesto nelle discussioni creative.

RiffBanks nasce con l'obiettivo di risolvere queste problematiche offrendo uno spazio digitale unificato pensato specificamente per le esigenze dei musicisti. La piattaforma integra tre funzionalità fondamentali: la gestione strutturata dei progetti musicali con versionamento degli asset, un sistema di comunicazione in tempo reale contestualizzato per ogni brano e una bacheca per il recruiting di musicisti (denominata "Gig Economy").

Il target principale dell'applicazione comprende musicisti amatoriali e semi-professionali, band in fase di formazione o consolidamento, e session musician alla ricerca di opportunità di collaborazione. L'interfaccia è stata progettata con un approccio mobile-first, riconoscendo che gran parte delle interazioni tra musicisti avviene in mobilità, durante le prove o negli spostamenti.

Capitolo 2

Requisiti

2.1 Analisi dell'utenza

Per comprendere a fondo le esigenze degli utenti finali, sono state definite tre personas rappresentative che incarnano i principali profili di utilizzo della piattaforma.

La prima persona è **Marco**, un chitarrista che suona stabilmente in una band rock composta da quattro elementi. Marco necessita di uno strumento efficace per condividere demo e ricevere feedback dai compagni di band, oltre a voler mantenere traccia delle diverse iterazioni di ogni brano durante il processo creativo. La sua frustrazione principale riguarda la dispersione delle informazioni: “Abbiamo tre gruppi WhatsApp diversi e non riesco mai a trovare i file quando mi servono”.

La seconda persona è **Giulia**, una cantautrice che lavora principalmente in autonomia sulla scrittura di testi e melodie. Giulia cerca regolarmente musicisti a chiamata per le fasi di registrazione dei suoi brani e ha bisogno di strumenti che la aiutino a superare i momenti di blocco creativo. Si chiede spesso dove poter trovare un batterista disponibile per una sessione di registrazione senza dover contattare decine di conoscenti.

La terza persona è **Alessandro**, un bassista freelance che lavora come musicista a chiamata per diverse band e progetti. Alessandro è costantemente alla ricerca di nuove opportunità di collaborazione e necessita di un modo efficiente per presentare il proprio profilo artistico. Attualmente deve monitorare numerose piattaforme e gruppi social per trovare offerte di lavoro, con un notevole dispendio di tempo.

2.2 Requisiti Utente

Dall'analisi dell'utenza emergono i requisiti che l'applicazione deve soddisfare dal punto di vista dell'esperienza utente. Gli utenti devono poter gestire un'area riservata personale contenente il proprio profilo con le preferenze musicali, inclusi gli strumenti suonati e i generi di riferimento. Devono poter creare nuove band e gestirne i membri attraverso un sistema di inviti basato su codici univoci, eliminando la necessità di condividere indirizzi email o altri dati personali.

La gestione dei contenuti musicali rappresenta il cuore dell'applicazione: gli utenti devono poter caricare e organizzare file audio e immagini associandoli a specifici brani, comunicare in tempo reale con gli altri membri della band nel contesto di ciascuna canzone, e ricevere notifiche tempestive per nuovi messaggi e attività rilevanti. Per quanto riguarda l'aspetto sociale, gli utenti devono poter cercare opportunità di collaborazione nella bacheca pubblica e candidarsi a quelle di interesse.

2.3 Requisiti Funzionali

2.3.1 Autenticazione e Gestione Utente

- Registrazione di nuovi utenti
- Login sicuro con generazione di token JWT
- Gestione del profilo personale tramite wizard di onboarding per la configurazione iniziale

2.3.2 Gestione delle Band

- Creazione di nuove band con generazione automatica di codice invito univoco (formato: XX-XXX-000)
- Condivisione del codice invito con potenziali membri
- Possibilità di unirsi a una band inserendo il codice nell'apposita sezione
- Gestione automatica dei ruoli:
 - Amministratori
 - Membri semplici

2.3.3 Progetti Musicali

- Operazioni CRUD complete sulle canzoni
- Workflow di stato per ogni canzone:
 1. Idea
 2. In Progress
 3. Mix
 4. Master
- Tipologie di asset associabili:
 - File audio caricati dall'utente
 - Immagini di riferimento
- Sistema di voto sugli asset:
 - Possibilità per i membri della band di esprimere preferenze
 - Conteggio aggiornato in tempo reale

2.3.4 Comunicazione

- Chat integrata per ogni canzone con:
 - Messaggi degli utenti
 - Notifiche di sistema (upload e altre attività significative)

2.3.5 Modulo Gig Economy

- Pubblicazione di annunci di due tipologie:
 - **Member:** posizioni permanenti nella band
 - **Session:** collaborazioni temporanee su specifici progetti
- Candidatura degli utenti agli annunci

2.4 Requisiti Non Funzionali

2.4.1 Usabilità

- Interfaccia intuitiva e user-friendly
- Design ottimizzato per dispositivi mobili (approccio mobile-first)

2.4.2 Accessibilità (WCAG 2.1 livello AA)

- Supporto per screen reader tramite attributi ARIA appropriati
- Touch target di dimensioni minime 44x44 pixel per dispositivi touch

2.4.3 Sicurezza

- Autenticazione tramite token JWT stateless con scadenza configurabile
- Password memorizzate con hashing bcrypt (10 salt rounds)
- Validazione di tutti gli input utente per prevenire injection e altri attacchi comuni

2.4.4 Performance e Scalabilità

- Tempi di risposta delle API inferiori a 200 millisecondi
- Architettura stateless del backend per scalabilità orizzontale
- Aggiornamenti in tempo reale (chat, notifiche, conteggio voti) tramite WebSocket con Socket.io

Capitolo 3

Design

3.1 Metodologia

Il sistema si basa su un database MongoDB organizzato in sei collezioni principali che modellano il dominio applicativo:

- **users**: memorizza i profili dei musicisti con le relative credenziali di autenticazione e le preferenze musicali espresse in termini di strumenti suonati e generi di riferimento.
- **bands**: rappresenta i gruppi di lavoro, contenendo il nome, il genere musicale, la località geografica, il codice invito univoco e l'elenco dei membri con i rispettivi ruoli.
- **songs**: archivia i progetti musicali associati a ciascuna band, con metadati quali titolo, BPM, genere e stato di avanzamento nel workflow produttivo.
- **assets**: contiene i file e i contenuti testuali associati alle canzoni, includendo il riferimento all'uploader, la tipologia (audio, immagine o testo), l'eventuale URL del file e l'array dei voti ricevuti.
- **messages**: gestisce la chat ibrida che combina messaggi degli utenti e log di sistema.
- **gigs**: gestisce gli annunci della bacheca Gig Economy con le relative candidature.

Le relazioni tra le entità seguono un modello orientato ai documenti tipico di MongoDB. Un utente può appartenere a molteplici band attraverso l'array di membri embedded in ciascuna band. Ogni band contiene multiple canzoni, referenziate tramite il campo **bandId**. A sua volta, ogni canzone può avere associati numerosi asset e messaggi, collegati tramite il campo **songId**. Gli annunci gig sono pubblicati da una band specifica e raccolgono candidature da parte degli utenti interessati.

La struttura del database e le relazioni tra le entità sono rappresentate nel diagramma ER in Figura 3.1.

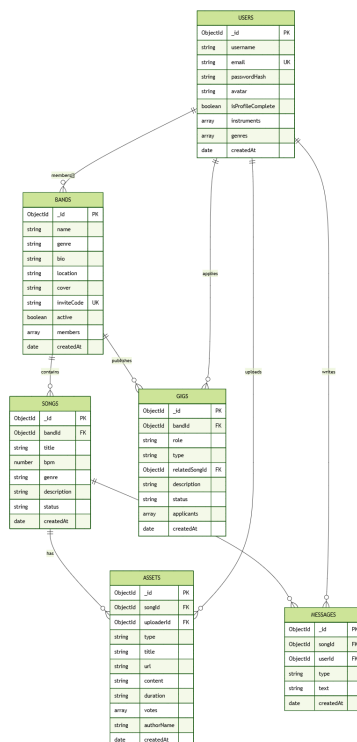


Figura 3.1: Diagramma ER del database

L'architettura generale segue il pattern REST per le operazioni CRUD, affiancato dal protocollo WebSocket per le funzionalità che richiedono aggiornamenti in tempo reale. Il frontend comunica con il backend attraverso chiamate HTTP per le operazioni transazionali e mantiene una connessione WebSocket persistente per ricevere notifiche push relative a nuovi messaggi, aggiornamenti dei voti e altre attività collaborative.

3.2 Architettura delle Interfacce Utente

3.2.1 Design System e Fonti di Ispirazione

Prima dello sviluppo delle interfacce, è stato definito uno stile visivo distintivo per il progetto. Le principali fonti di ispirazione sono state:

- **Teenage Engineering:** brand noto per il design minimalista e funzionale dei propri dispositivi audio, caratterizzato da un'estetica industriale con accenti di colore vibranti e tipografia bold.
- **Nothing:** azienda di consumer electronics riconoscibile per l'approccio visivo basato su trasparenze, elementi geometrici e un linguaggio grafico pulito con forte identità.

Da queste influenze è stata creata una mood board contenente palette colori, font di riferimento e screenshot dei prodotti di ispirazione, insieme alla definizione delle funzionalità che ogni schermata principale doveva offrire.

3.2.2 Processo di Creazione dei Mockup

Per la realizzazione dei mockup è stato adottato un approccio non convenzionale: invece di utilizzare strumenti tradizionali come Figma, si è scelto di sperimentare con **Gemini 3.0 Flash** utilizzando la funzionalità Canvas. Partendo dalla mood board e dalle specifiche funzionali delle schermate, sono state condotte diverse iterazioni di prompt e raffinamenti successivi che hanno portato alla generazione dei mockup finali.

Questo approccio ha permesso di:

- Esplorare rapidamente molteplici varianti stilistiche

- Iterare velocemente sul feedback visivo
- Mantenere coerenza con il design system definito

3.2.3 Mockup

Di seguito i mockup delle schermate principali dell'applicazione:

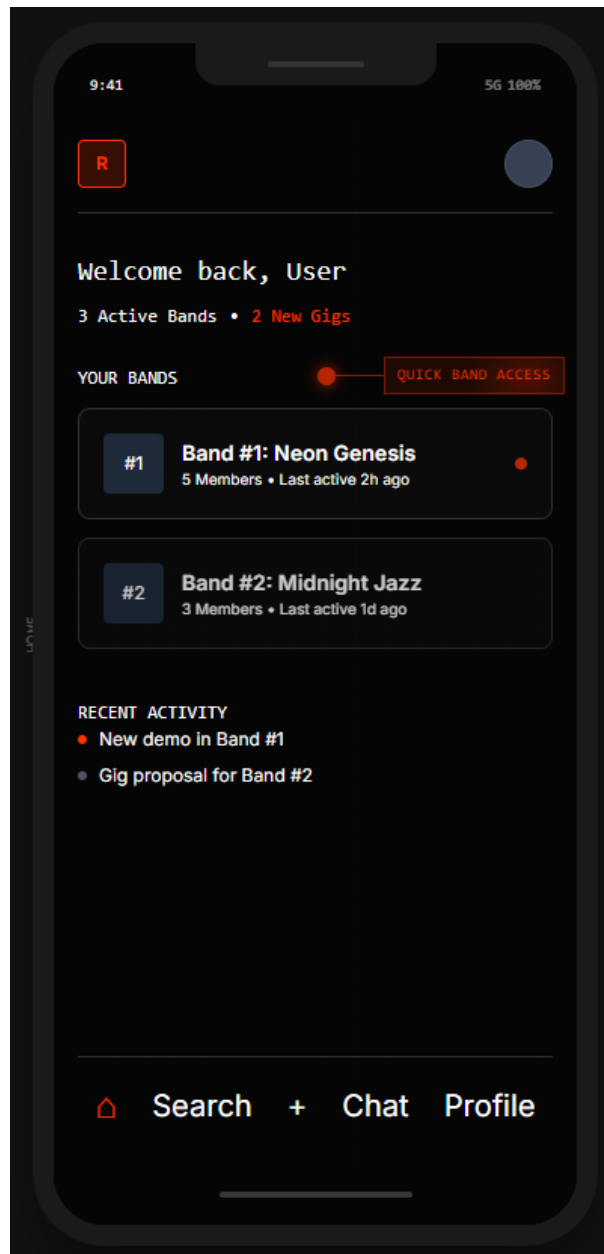


Figura 3.2: Dashboard principale con elenco delle band

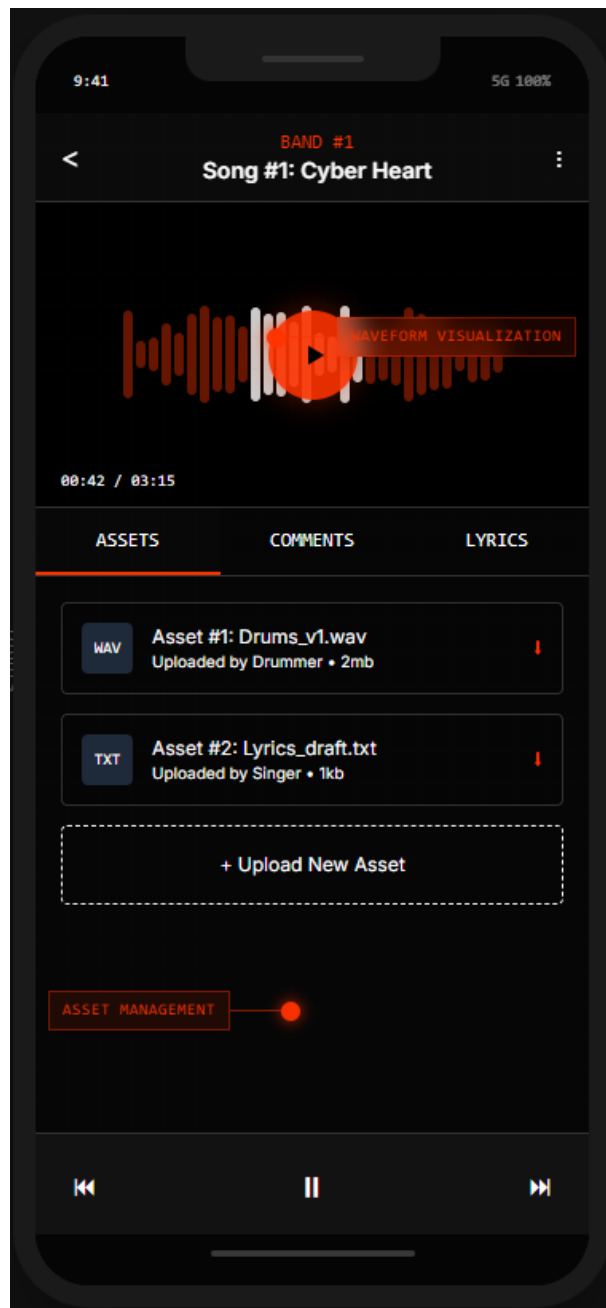


Figura 3.3: Vista dettaglio canzone con player e asset

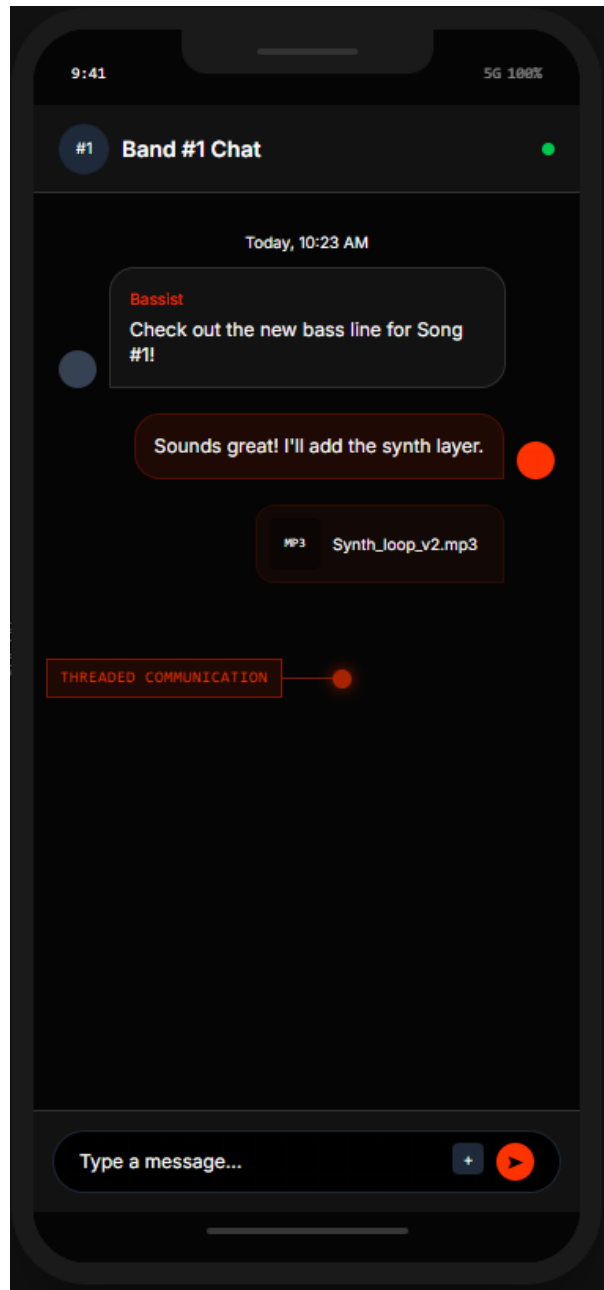


Figura 3.4: Chat integrata per la comunicazione contestuale

3.2.4 Behavior-Driven Development

Per definire i casi d'uso e guidare la progettazione dei mockup è stato adottato un approccio **Behavior-Driven Development (BDD)**. I requisiti funzionali sono stati formalizzati in scenari Gherkin, che descrivono il comportamento atteso dell'applicazione dal punto di vista dell'utente.

Questa metodologia ha permesso di:

- Chiarire i flussi utente prima della progettazione visiva
- Creare un linguaggio condiviso tra requisiti e implementazione
- Definire criteri di accettazione verificabili per ogni funzionalità

Di seguito alcuni esempi delle feature definite:

```

1 Feature: User onboarding
2
3 Scenario: Account creation and onboarding completion

```

```

4   Given I am a new user on the registration page
5   When I enter the name "Mario", email and password
6   Then I am redirected to the onboarding wizard
7   When I select an instrument
8   And I select a genre
9   Then my profile is saved
10  And I access the empty dashboard

```

Listing 3.1: Feature: Registrazione e onboarding

```

1 Feature: Band management
2
3   Scenario: Creating a new band
4     Given I am a logged-in user without a band
5     When I click on "Create New Band"
6     And I enter the name and the genre
7     Then a new band is created
8     And I am assigned the role "Admin"
9     And a unique invite code is generated

```

Listing 3.2: Feature: Creazione di una band

```

1 Feature: Join band by invite code
2
3   Scenario: Join an existing band using invite code
4     Given I am a logged-in user without a band
5     When I enter the invite code "RC-99X-DEV"
6     And I click on "Join band"
7     Then I am added to the band members list
8     And I see the band dashboard

```

Listing 3.3: Feature: Join band tramite codice invito

3.2.5 Viste dell'Applicazione

L'applicazione si articola nelle seguenti viste principali:

- **Autenticazione:** form per login e registrazione
- **Onboarding:** wizard di configurazione iniziale del profilo
- **Dashboard:** panoramica delle band e accesso alla bacheca gig
- **Dettaglio Band:** informazioni, membri e impostazioni della band
- **Lista Canzoni:** elenco dei brani con indicatori di stato
- **Dettaglio Canzone:** player audio, asset e chat contestuale
- **Gig:** bacheca annunci con filtri di ricerca

3.2.6 Screenshot dell'Applicazione

Di seguito alcune schermate dell'applicazione finale, che mostrano l'implementazione del design system definito in fase di progettazione.

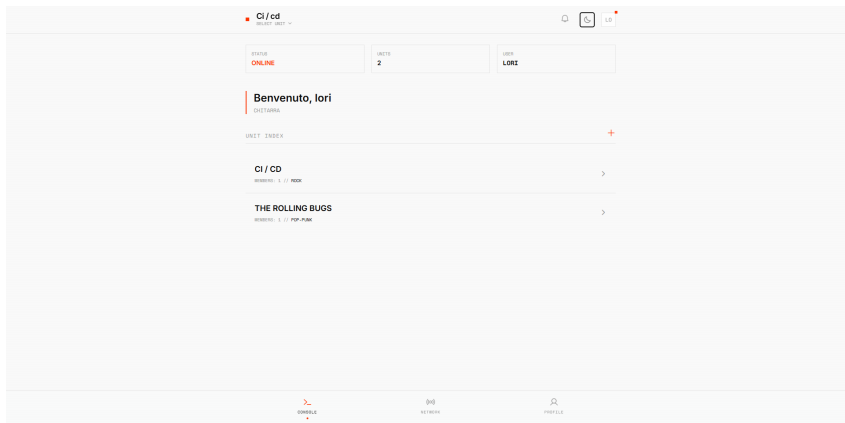


Figura 3.5: Dashboard principale in tema chiaro. Mostra lo stato dell'utente, il numero di band (Units) a cui appartiene e l'elenco delle band con genere e numero di membri. La bottom navigation permette l'accesso rapido a Console, Network (Gig Economy) e Profilo.

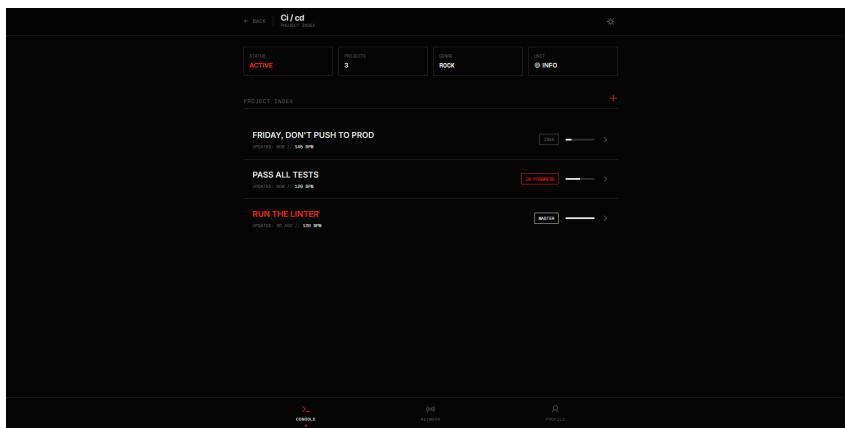


Figura 3.6: Lista canzoni di una band in tema scuro. Ogni brano mostra titolo, data di aggiornamento, BPM e stato nel workflow (Idea, In Progress, Mix, Master). La barra di progresso visualizza lo stato di avanzamento del brano.

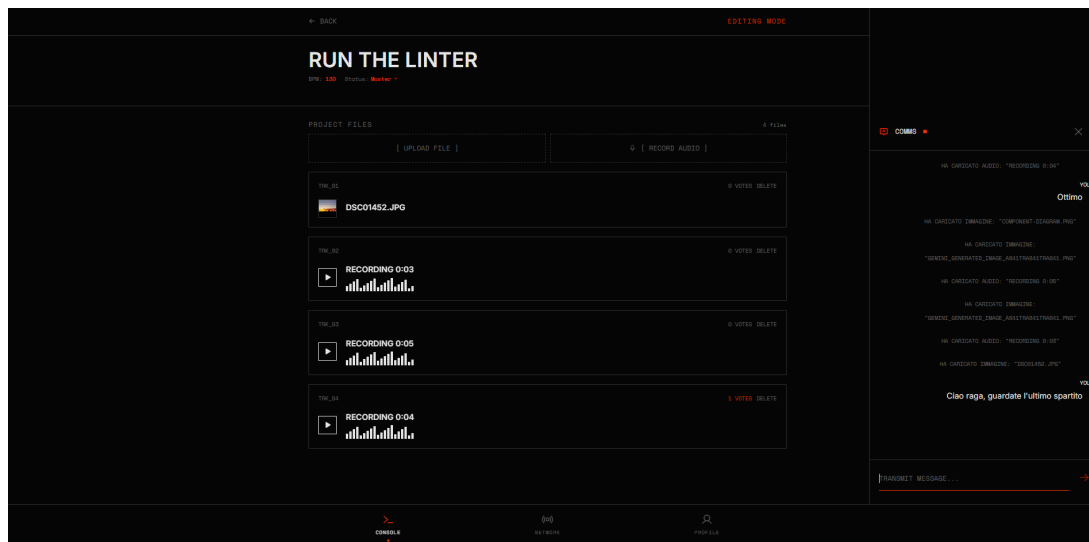


Figura 3.7: Dettaglio canzone con pannello chat. A sinistra gli asset caricati (immagini e registrazioni audio) con sistema di voto; a destra la chat contestuale che mostra messaggi utente e notifiche di sistema per upload di file. Il pannello COMMS permette la comunicazione real-time tra i membri della band.

Post_Gig
✕

UNIT •

CI / CD

GIG_TITLE •

LEAD GUITARIST NEEDED

TYPE •

BAND MEMBER
PERMANENT POSITION

SESSION
PROJECT OR GIG

ROLE •

SELECT_ROLE

GENRE

LOCATION

ROCK, JAZZ...

CITY, REMOTE...

COMPENSATION

REVENUE SHARE, \$200/GIG, UNPAID

DESCRIPTION

Describe the opportunity...

CANCEL

POST_GIG

Figura 3.8: Modal di creazione annuncio Gig. Permette di specificare la band, il titolo, la tipologia (Band Member per posizioni permanenti o Session per collaborazioni temporanee), il ruolo richiesto, genere, località, compenso e descrizione dell'opportunità.

Capitolo 4

Tecnologie

Lo stack tecnologico adottato è il **MEVN** (MongoDB, Express, Vue.js, Node.js), arricchito da Socket.io per le funzionalità real-time e containerizzato con Docker per il deployment.

Questa combinazione offre diversi vantaggi:

- **JavaScript full-stack:** un unico linguaggio per frontend, backend e query database (MongoDB), riducendo il context-switching e facilitando la condivisione di logica tra i layer
- **Architettura asincrona:** Node.js gestisce efficacemente connessioni WebSocket simultanee per la chat real-time
- **Schema flessibile:** MongoDB si adatta naturalmente a dati eterogenei come asset musicali, metadati e messaggi
- **Reattività:** Vue.js con Composition API permette interfacce responsive con stato condiviso tra componenti
- **Portabilità:** Docker garantisce ambienti consistenti tra sviluppo e produzione

4.1 Containerizzazione

- **Docker:** containerizzazione dell'intera applicazione per deployment riproducibile e isolato
- **Docker Compose:** orchestrazione multi-container (MongoDB, Express, nginx) con gestione delle dipendenze

4.2 Backend

4.2.1 Runtime e Framework

- **Node.js 18+:** modello asincrono non bloccante, adatto ad applicazioni con elevata concorrenza
- **Express 4:** framework web per API RESTful con gestione middleware

4.2.2 Persistenza Dati

- **MongoDB:** database documentale NoSQL, flessibile per dati musicali e collaborativi
- **Mongoose:** ODM con validazione schema, middleware e query builder

4.2.3 Real-time

- **Socket.io:** WebSocket con fallback automatico su polling, organizzazione in “room” per canzone

4.2.4 Autenticazione e Sicurezza

- **jsonwebtoken**: autenticazione stateless tramite JWT
- **bcryptjs**: hashing password con algoritmo bcrypt (10 salt rounds)

4.2.5 Upload File

- **Multer**: parsing multipart/form-data con limite 50MB per file
- Formati accettati: audio (mp3, wav, ogg, webm) e immagini (jpg, png, gif)

4.3 Frontend

4.3.1 Framework e Architettura

- **Vue.js 3** con Composition API e sintassi `<script setup>` per logica reattiva concisa
- **Pinia**: state management centralizzato per autenticazione e connessione Socket.io
- **Vue Router**: routing client-side con navigation guard e lazy loading delle viste

4.3.2 Build e Sviluppo

- **Vite**: Hot Module Replacement istantaneo, proxy API integrato per lo sviluppo

4.3.3 UI e Styling

- **Tailwind CSS**: approccio utility-first per styling inline
- **Lucide Vue Next**: libreria icone
- **Axios**: client HTTP con interceptor per JWT e gestione errori autenticazione

Capitolo 5

Codice

5.1 Struttura del Progetto

Il progetto adotta una struttura monorepo gestita attraverso npm workspace, con due package principali:

- **client:** contenente il frontend in Vue
- **server:** backend Express

Questa organizzazione permette di condividere configurazioni e script a livello di root mantenendo al contempo una chiara separazione di responsabilità.

5.1.1 Client

La cartella client contiene l'applicazione Vue organizzata secondo le convenzioni del framework:

- **components:** ospita i componenti riutilizzabili come AudioPlayer, ChatPanel, GigCard e i vari modal
- **views:** contiene i componenti che rappresentano le pagine dell'applicazione, mappati alle route definite nel router
- **stores:** contiene i Pinia store per auth e socket
- **services:** contiene il modulo api che centralizza tutte le chiamate HTTP
- **routes:** definisce tutte le route e i navigation guard

5.1.2 Server

La cartella server contiene tutto il backend del progetto ed è organizzata nel seguente modo:

- **models:** contiene gli schemi di Mongoose per le diverse collezioni del DB
- **controllers:** implementa la logica applicativa per ciascuna risorsa
- **routes:** definisce tutti gli endpoint e li collega al controller
- **middleware:** ospita i moduli per autenticazione e upload
- **services:** contiene la logica di business riutilizzabile
- **socket:** gestisce gli handler per gli eventi WebSocket

5.1.3 Diagramma dei Componenti

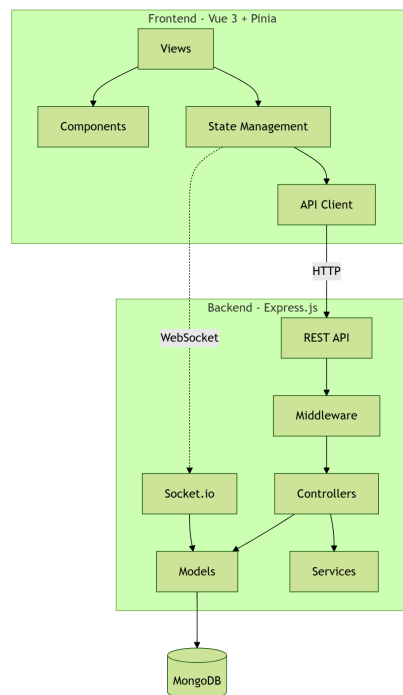


Figura 5.1: Diagramma dei componenti dell'architettura

5.2 Pattern e Implementazioni Rilevanti

L'architettura complessiva segue un **pattern a strati** che garantisce separazione delle responsabilità:

1. **Route:** validazione preliminare e applicazione middleware
2. **Controller:** orchestrazione della logica applicativa
3. **Model:** accesso ai dati tramite Mongoose

5.2.1 Controller Backend: Gestione Band

Il controller `bandController.js` implementa le operazioni CRUD sulle band. Di seguito un estratto della funzione di creazione:

```
1 // server/controllers/bandController.js
2 exports.create = async (req, res) => {
3   try {
4     const { name, genre, bio, location, instrument } = req.body;
5
6     // Generate unique invite code
7     let inviteCode;
8     let isUnique = false;
9     while (!isUnique) {
10       inviteCode = Band.generateInviteCode(name);
11       const existing = await Band.findOne({ inviteCode });
12       if (!existing) isUnique = true;
13     }
14
15     const band = new Band({
16       name, genre, bio, location, inviteCode,
17       members: [{
18         userId: req.userId,
19         role: 'Admin',
20         instrument: instrument || req.user.instruments?.[0] || '',
21         joinedAt: new Date()
22       }]
23     });
```

```

23 });
24
25 await band.save();
26 await band.populate('members.userId', 'username avatar');
27 res.status(201).json(band);
28 } catch (err) {
29   res.status(500).json({ error: 'Failed to create band' });
30 }
31 };

```

Listing 5.1: Creazione di una band

La funzione join verifica l'esistenza della band e l'appartenenza dell'utente prima di aggiungerlo:

```

1 // server/controllers/bandController.js
2 exports.join = async (req, res) => {
3   const { inviteCode, instrument } = req.body;
4
5   const band = await Band.findOne({
6     inviteCode: inviteCode.toUpperCase(),
7     active: true
8   });
9
10  if (!band) {
11    return res.status(404).json({ error: 'Invalid invite code' });
12  }
13
14  if (band.isMember(req.userId)) {
15    return res.status(400).json({ error: 'You are already a member' });
16  }
17
18  band.members.push({
19    userId: req.userId,
20    role: 'Member',
21    instrument: instrument || req.user.instruments?.[0] || '',
22    joinedAt: new Date()
23  });
24
25  await band.save();
26  res.json(band);
27 };

```

Listing 5.2: Join a una band tramite codice invito

5.2.2 State Management Frontend: Auth Store

Lo store Pinia per l'autenticazione utilizza la Composition API:

```

1 // client/src/stores/auth.ts
2 export const useAuthStore = defineStore('auth', () => {
3   const user = ref<any>(null)
4   const loading = ref(true)
5
6   const isAuthenticated = computed(() => !!user.value)
7   const needsOnboarding = computed(() =>
8     user.value && !user.value.isProfileComplete
9   )
10
11  async function checkAuth() {
12    const token = localStorage.getItem('token')
13    if (token) {
14      try {
15        const res = await authAPI.me()
16        user.value = res.data
17      } catch (err) {
18        localStorage.removeItem('token')
19      }
20    }
21    loading.value = false
22  }
23
24  async function login(email: any, password: any) {
25    const res = await authAPI.login({ email, password })

```

```

26   localStorage.setItem('token', res.data.token)
27   user.value = res.data.user
28   return res.data.user
29 }
30
31 function logout() {
32   localStorage.removeItem('token')
33   user.value = null
34 }
35
36 return { user, loading, isAuthenticated, needsOnboarding,
37         checkAuth, login, logout }
38 })

```

Listing 5.3: Auth Store con Pinia

5.2.3 Real-time: Socket.io

La gestione Socket.io lato server (server/socket/index.js) implementa:

- **Autenticazione** via middleware JWT
- **Room per canzone:** ogni client si unisce a `song:{songId}`
- **Broadcast messaggi:** eventi `send_message` → `new_message`

```

1 // server/socket/index.js
2 socket.on('join_room', async (songId) => {
3   const song = await Song.findById(songId);
4   const band = await Band.findById(song.bandId);
5
6   const isMember = band.members.some(
7     m => m.userId.toString() === socket.userId
8   );
9   if (!isMember) {
10    socket.emit('error', { message: 'Not a band member' });
11    return;
12  }
13
14  socket.join(`song:${songId}`);
15  socket.currentSongId = songId;
16 });
17
18 socket.on('send_message', async (data) => {
19   const { songId, text } = data;
20
21   const message = new Message({
22     songId,
23     userId: socket.userId,
24     type: 'user',
25     text: text.trim(),
26     readBy: [socket.userId]
27   });
28
29   await message.save();
30   await message.populate('userId', 'username profilePic');
31
32   io.to(`song:${songId}`).emit('new_message', messageData);
33 });

```

Listing 5.4: Gestione eventi Socket.io

5.2.4 Sistema di Voto

Il meccanismo di voto implementa un toggle like/unlike con notifica real-time:

```

1 // server/controllers/assetController.js
2 exports.vote = async (req, res) => {
3   const asset = await Asset.findById(req.params.id);
4
5   const voteIndex = asset.votes.findIndex(v => v.equals(req.userId));

```

```

6  const votedByMe = voteIndex === -1;
7
8  if (votedByMe) {
9    asset.votes.push(req.userId);    // Add vote
10 } else {
11   asset.votes.splice(voteIndex, 1); // Remove vote
12 }
13
14 await asset.save();
15
16 // Notify all clients in the room
17 req.io.to('song:${asset.songId}').emit('vote_update', {
18   assetId: asset._id,
19   votes: asset.votes.length,
20   action: votedByMe ? 'liked' : 'unliked'
21 });
22
23 res.json({ votes: asset.votes.length, votedByMe });
24 };

```

Listing 5.5: Sistema di voto sugli asset

5.2.5 Diagramma Comunicazione Real-time

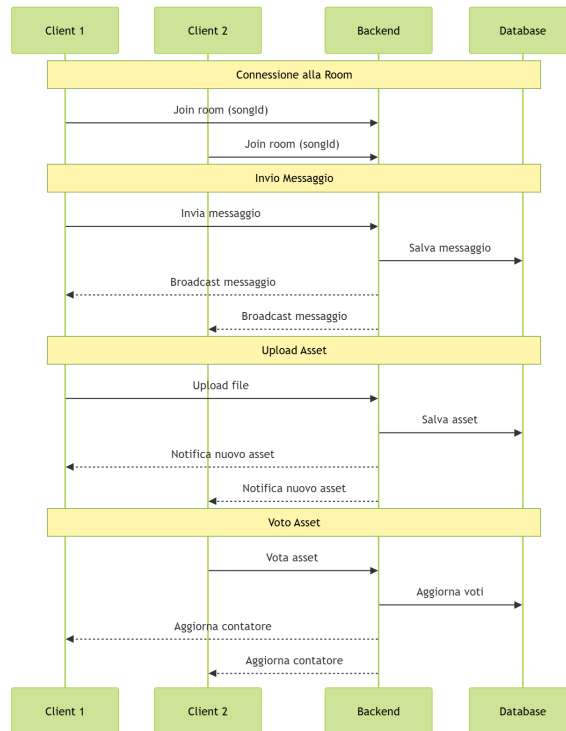


Figura 5.2: Sequence diagram della comunicazione real-time

Capitolo 6

Test

6.1 Test Tecnici

La verifica del corretto funzionamento dell'applicazione è stata condotta attraverso diverse tipologie di test. Il testing cross-browser ha verificato la compatibilità con i principali browser desktop (Chrome, Firefox, Edge) e i browser mobile su sistemi iOS e Android. L'approccio mobile-first adottato durante lo sviluppo ha garantito che l'esperienza su dispositivi touch fosse ottimale fin dalle prime fasi.

Il testing delle API è stato eseguito sistematicamente utilizzando Postman, costruendo una collection che copre tutti gli endpoint con casi di test per i flussi nominali e per le condizioni di errore. Particolare attenzione è stata dedicata alla verifica dei meccanismi di autenticazione, testando il comportamento con token validi, scaduti e malformati.

Il funzionamento real-time è stato verificato simulando scenari con connessioni multiple simultanee, confermando la corretta propagazione degli eventi di chat e delle notifiche di sistema a tutti i client iscritti alla room interessata.

6.2 Euristiche di Nielsen

Per garantire un'esperienza utente di qualità, l'interfaccia è stata progettata seguendo le dieci euristiche di usabilità di Nielsen:

1. **Visibilità dello stato del sistema:** l'applicazione fornisce feedback costante sulle operazioni in corso attraverso spinner di caricamento animati durante le chiamate API, indicatori di "typing" nella chat che mostrano quando altri utenti stanno scrivendo, badge numerici per le notifiche non lette, e lo stato di avanzamento di ogni canzone (Idea, In Progress, Mix, Master) sempre visibile nella lista brani.
2. **Corrispondenza tra sistema e mondo reale:** la terminologia adottata rispecchia il linguaggio comune nel mondo musicale (band, canzone, gig, session musician). Il workflow di produzione delle canzoni segue le fasi reali del processo creativo, dalla nascita dell'idea fino al master finale. Le icone utilizzate (Lucide) sono intuitive e convenzionali.
3. **Controllo e libertà per l'utente:** la navigazione è completamente libera grazie alla bottom navigation sempre accessibile. Gli utenti possono annullare i propri voti sugli asset con un semplice tap (toggle like/unlike), e le azioni distruttive richiedono conferma attraverso modal dedicati, prevenendo operazioni accidentali.
4. **Consistenza e standard:** l'applicazione mantiene coerenza visiva attraverso il design system ispirato a Teenage Engineering e Nothing. Tutti i form di creazione (band, canzone, gig) seguono lo stesso pattern di interazione. Il formato del codice invito (XX-XXX-000) è uniforme in tutta la piattaforma.
5. **Prevenzione dall'errore:** i form implementano validazione lato client per prevenire l'invio di dati incompleti o malformati. Prima di eliminare elementi viene sempre richiesta conferma. Il codice invito accetta input case-insensitive, riducendo gli errori di digitazione.

6. **Riconoscimento più che ricordo:** la dashboard presenta tutte le band dell'utente in modo immediato, senza necessità di navigazione. La lista canzoni mostra lo stato di ciascun brano con indicatori visivi. I filtri nella sezione Gig Economy permettono di restringere la ricerca senza dover ricordare criteri precedenti.
7. **Flessibilità ed efficienza:** il wizard di onboarding guida i nuovi utenti nella configurazione iniziale del profilo, mentre gli utenti esperti possono saltare direttamente alle funzionalità. Il sistema di codice invito permette di unirsi a una band con una singola azione. La chat contestualizzata per canzone evita la dispersione delle conversazioni.
8. **Estetica e progettazione minimalista:** il design minimale, ispirato all'estetica industriale di Teenage Engineering, elimina elementi superflui. L'approccio mobile-first ha imposto rigore nella selezione delle informazioni da mostrare, privilegiando la chiarezza alla completezza.
9. **Facilità di riconoscimento, diagnosi e risoluzione delle situazioni di errore:** i messaggi di errore sono descrittivi e suggeriscono possibili soluzioni (es. "Codice invito non valido - verifica di averlo digitato correttamente"). Le notifiche toast forniscono feedback immediato sulle operazioni completate o fallite.
10. **Documentazione:** il wizard di onboarding funge da tutorial integrato per i nuovi utenti. L'interfaccia è stata progettata per essere autoesplicativa, riducendo al minimo la necessità di documentazione esterna.

6.3 Test di Usabilità

Un gruppo ristretto di utenti ha partecipato a sessioni di test di usabilità, fornendo feedback iterativo durante lo sviluppo. I partecipanti sono stati selezionati in modo da rappresentare le tre personas identificate in fase di analisi. A ciascun tester è stato assegnato un insieme di task da completare senza alcuna guida preliminare, osservando come interagivano con l'applicazione al primo utilizzo.

- **Profilo "Marco" (chitarrista in band stabile):** ai tester di questo profilo è stato chiesto di creare una nuova band, generare e condividere il codice invito, aggiungere una canzone caricando un file audio demo, esprimere preferenze sugli asset tramite il sistema di voto, e comunicare con i compagni di band attraverso la chat contestuale. I partecipanti hanno completato tutti i task senza difficoltà significative, apprezzando in particolare l'immediatezza del sistema di invito tramite codice.
- **Profilo "Giulia" (cantautrice solista):** a questo gruppo è stato richiesto di pubblicare un annuncio nella sezione Gig Economy per cercare un batterista per una sessione di registrazione, consultare eventuali candidature ricevute, e aggiornare il proprio profilo con strumenti suonati e generi musicali. I tester hanno trovato intuitivo il flusso di creazione degli annunci, suggerendo come miglioramento l'aggiunta di filtri più granulari per la ricerca.
- **Profilo "Alessandro" (musicista freelance):** i partecipanti hanno dovuto esplorare la bacheca Gig Economy alla ricerca di opportunità, filtrare gli annunci per strumento e genere, e candidarsi a quelli di interesse. È emersa l'utilità della distinzione tra annunci "Member" (posizioni permanenti) e "Session" (collaborazioni temporanee), che permette di identificare rapidamente le opportunità più rilevanti.

In generale, i feedback raccolti hanno confermato l'efficacia delle scelte di design adottate. Le osservazioni degli utenti hanno riguardato principalmente aspetti stilistici minori e suggerimenti per funzionalità aggiuntive, mentre la struttura e i flussi di navigazione sono stati giudicati chiari e coerenti.

6.4 Validazione e Accessibilità

L'applicazione è stata sottoposta a verifica di conformità agli standard web. La validazione del markup HTML tramite il W3C Validation Service non ha evidenziato errori critici. Per quanto riguarda l'accessibilità, sono state implementate le linee guida WCAG 2.1 livello AA:

- Attributi ARIA appropriati per supportare gli screen reader

- Touch target di dimensioni minime 44x44 pixel per tutti gli elementi interattivi
- Contrasto cromatico sufficiente tra testo e sfondo
- Navigazione completamente accessibile da tastiera

I test con Lighthouse hanno restituito punteggi soddisfacenti nelle categorie Performance (75), Accessibility (88), Best Practices (96) e SEO (83), confermando la qualità complessiva dell'implementazione frontend.

Capitolo 7

Deployment

Il deployment dell'applicazione è stato progettato utilizzando **Docker** per garantire portabilità, riproducibilità e semplicità di installazione.

7.1 Architettura dei Container

L'applicazione è composta da tre container orchestrati tramite Docker Compose:

Container	Immagine	Porta	Ruolo
riffbanks-mongodb	mongo:latest	27017	Database documentale
riffbanks-server	Node 20 Alpine	3333	API Express + Socket.io
riffbanks-client	nginx Alpine	80	Frontend Vue + Reverse Proxy

Tabella 7.1: Architettura dei container Docker

7.2 Build Multi-Stage del Frontend

Il Dockerfile del client utilizza un approccio multi-stage per ottimizzare l'immagine finale:

```
1 # client/Dockerfile
2 FROM node:20-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 FROM nginx:alpine
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 COPY --from=builder /app/dist /usr/share/nginx/html
12 EXPOSE 80
13 CMD ["nginx", "-g", "daemon off;"]
```

Listing 7.1: Dockerfile multi-stage del client

Lo stage di build compila l'applicazione Vue, mentre lo stage di produzione serve solo i file statici risultanti tramite nginx.

7.3 Configurazione Nginx

Il reverse proxy nginx gestisce il routing tra frontend e backend:

```
1 # client/nginx.conf
2 server {
3     listen 80;
4     root /usr/share/nginx/html;
5
6     # SPA fallback per Vue Router
```

```

7   location / {
8       try_files $uri $uri/ /index.html;
9   }
10
11  # Proxy API verso backend
12  location /api {
13      proxy_pass http://server:3333;
14      proxy_http_version 1.1;
15      proxy_set_header Upgrade $http_upgrade;
16      proxy_set_header Connection 'upgrade';
17  }
18
19  # Proxy WebSocket per Socket.io
20  location /socket.io {
21      proxy_pass http://server:3333;
22      proxy_http_version 1.1;
23      proxy_set_header Upgrade $http_upgrade;
24      proxy_set_header Connection "upgrade";
25      proxy_read_timeout 86400;
26  }
27
28  # Proxy file uploads
29  location /uploads {
30      proxy_pass http://server:3333;
31  }
32 }

```

Listing 7.2: Configurazione nginx

La configurazione include inoltre:

- **Gzip compression** per ridurre il traffico di rete
- **Security headers:** X-Frame-Options, X-Content-Type-Options, X-XSS-Protection
- **Cache 1 anno** per asset statici (js, css, immagini, font)

7.4 Persistenza dei Dati

Docker Compose definisce due volumi nominati per la persistenza:

```

1 # docker-compose.yml
2 volumes:
3   mongodb_data:
4     name: riffbanks-mongodb-data    # Dati MongoDB
5   uploads_data:
6     name: riffbanks-uploads-data    # File caricati dagli utenti

```

Listing 7.3: Volumi Docker Compose

7.5 Variabili d'Ambiente

La configurazione è esternalizzata tramite variabili d'ambiente:

Variabile	Default	Descrizione
NODE_ENV	production	Ambiente di esecuzione
PORT	3333	Porta del server Express
MONGODB_URI	mongodb://mongodb:27017/riffbanks	Connection string
JWT_SECRET	riffbanks	Chiave per firma token JWT
JWT_EXPIRES_IN	7d	Scadenza token
CLIENT_URL	http://localhost	URL frontend per CORS

Tabella 7.2: Variabili d'ambiente

7.6 Health Checks

Ogni container implementa un health check per il monitoraggio automatico:

MongoDB:

```
1 healthcheck:
2   test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
3   interval: 30s
4   timeout: 10s
5   retries: 3
```

Listing 7.4: Health check MongoDB

Server Express (endpoint /api/health):

```
1 app.get('/api/health', (req, res) => {
2   res.json({ status: 'ok', message: 'RiffBank API is running' });
3 });
```

Listing 7.5: Endpoint health check

Client nginx:

```
1 HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
2   CMD wget --no-verbose --tries=1 --spider http://localhost/ || exit 1
```

Listing 7.6: Health check nginx

7.7 Avvio dell'Applicazione

Per avviare l'intero stack:

```
1 # Clonare il repository
2 git clone https://github.com/LoryBug/RiffBanks.git
3 cd RiffBanks
4
5 # Configurare JWT_SECRET (opzionale)
6 export JWT_SECRET=your-secret-key
7
8 # Avviare i container
9 docker-compose up -d
10
11 # Verificare lo stato
12 docker-compose ps
```

Listing 7.7: Comandi per l'avvio

L'applicazione sarà disponibile su <http://localhost> (porta 80).

7.8 Dipendenze tra Servizi

Docker Compose gestisce l'ordine di avvio:

1. **MongoDB** si avvia per primo
2. **Server** attende che MongoDB sia healthy (`condition: service_healthy`)
3. **Client** attende che il server sia disponibile

Capitolo 8

Conclusioni

Il progetto RiffBanks ha raggiunto la maggior parte degli obiettivi prefissati, portando a termine lo sviluppo di una piattaforma funzionante che risponde alle esigenze identificate in fase di analisi.

Il prodotto finale offre un'esperienza d'uso coerente e intuitiva, permettendo ai musicisti di gestire le proprie band, organizzare i progetti musicali, comunicare in tempo reale e cercare opportunità di collaborazione attraverso un'unica interfaccia. L'approccio mobile-first ha garantito un'esperienza ottimale sui dispositivi che i musicisti utilizzano maggiormente nella loro quotidianità.

8.1 Sviluppi Futuri

Per quanto riguarda le funzionalità che interessano gli utenti, un'evoluzione futura del sistema potrà concentrarsi su possibili estensioni quali:

- **Player audio avanzato:** integrazione di funzionalità di editing audio base (trim, fade) e visualizzazione della forma d'onda per una migliore gestione degli asset musicali
- **Storage cloud:** migrazione dello storage dei file su servizi cloud (AWS S3, Cloudinary) per migliorare scalabilità e affidabilità
- **Sistema di recensioni:** possibilità per i membri delle band di lasciare feedback sui collaboratori incontrati tramite la sezione Gig Economy

8.2 Commenti Finali

Lo sviluppo di questo progetto ha permesso di mettere in pratica le conoscenze acquisite durante il corso, affrontando le sfide tipiche dello sviluppo web moderno: dalla progettazione di API RESTful alla gestione della comunicazione real-time tramite WebSocket, dall'autenticazione JWT alla containerizzazione con Docker.

Particolarmente formativa è stata l'esperienza con lo stack MEVN, che ha richiesto di padroneggiare tecnologie diverse ma coerenti tra loro, dalla reattività di Vue.js con la Composition API alla flessibilità di MongoDB come database documentale.

Un aspetto interessante del processo di sviluppo è stato l'utilizzo di strumenti di intelligenza artificiale generativa per la creazione dei mockup, un approccio non convenzionale che ha permesso di esplorare nuove modalità di prototipazione rapida.

L'idea di RiffBanks nasce da un'esigenza concreta riscontrata nel mondo della musica amatoriale: la frammentazione degli strumenti di collaborazione. Questo ha permesso di sviluppare con maggiore consapevolezza le funzionalità necessarie, potendo valutare in prima persona l'utilità delle scelte implementative.